COMPONENTS IN LEPTOS

I. Lesson

What is a Component? 🤔

In Leptos, a component is just a Rust **function** that takes props as arguments and returns something that can be turned into a **view**.

Syntax specific to Leptos

```rust
use leptos::prelude::*;

#[component]
pub fn SimpleCounter() -> impl IntoView {
    let (value, set_value) = signal(0);

    let increment = move |_| set_value.update(|value| *value += 1);
    let decrement = move |_| set_value.update(|value| *value -= 1);
    let clear = move |_| set_value.set(0);

    view! {
        <div>
            <button on:click=clear>"Clear"</button>
            <button on:click=decrement>"-1"</button>
            <span>"Value: " {value} "!"</span>
            <button on:click=increment>"+1"</button>
        </div>
    }
}
```

Prelude

-> Avoid import everything (view! etc.)

-> Keeps code clean

-> Quicker to prototype

-> Standard pattern in Rust

#[component]

-> Attribute macro

-> tells the compiler to generate the necessary code to make the function reactive

-> Useful for accepting props

impl IntoView

-> It’s a trait

-> converts Rust types (like HTML elements, strings, signals, or even other components) into DOM renderable structures

-> "this fn produces something you can render into the page."

II. Exercices

Taking props for components

```rust
#[component]
pub fn BasicCounter(
    initial_value: i32,
    step: i32,
    title: String,
) -> impl IntoView {
```

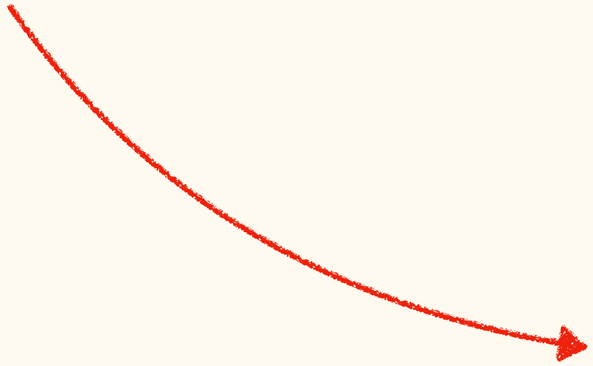
Component definition

.. in the view macro

```rust
<BasicCounter
    initial_value=5
    step=1
    title="Basic Counter".to_string()
/>
```

```rust
#[component]
pub fn BasicCounter(
    initial_value: i32,
    #[prop(default = 1)] step: i32,
    // This avoid to have "sth".to_string()
    #[prop(into)] title: String,
) -> impl IntoView {
```

With default props and into

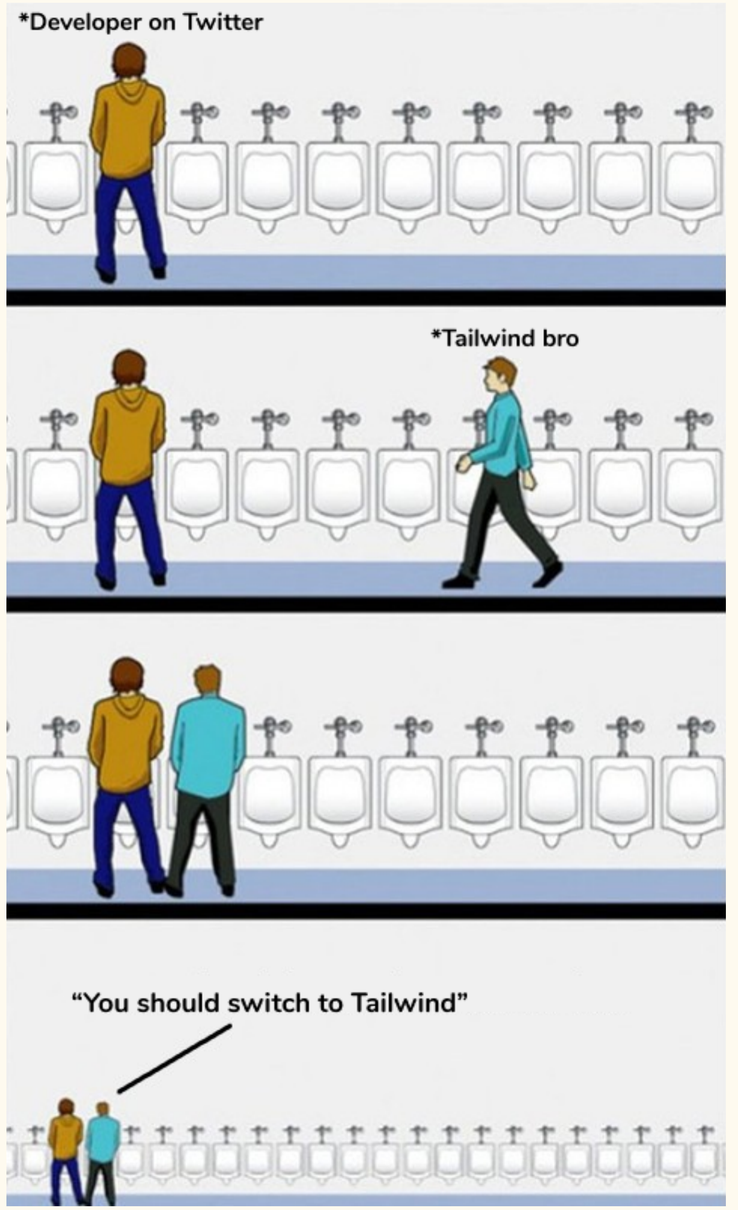


```rust
<BasicCounter
    initial_value=5
    title="Basic Counter"
/>
```

Exercice 1:
Counter Props

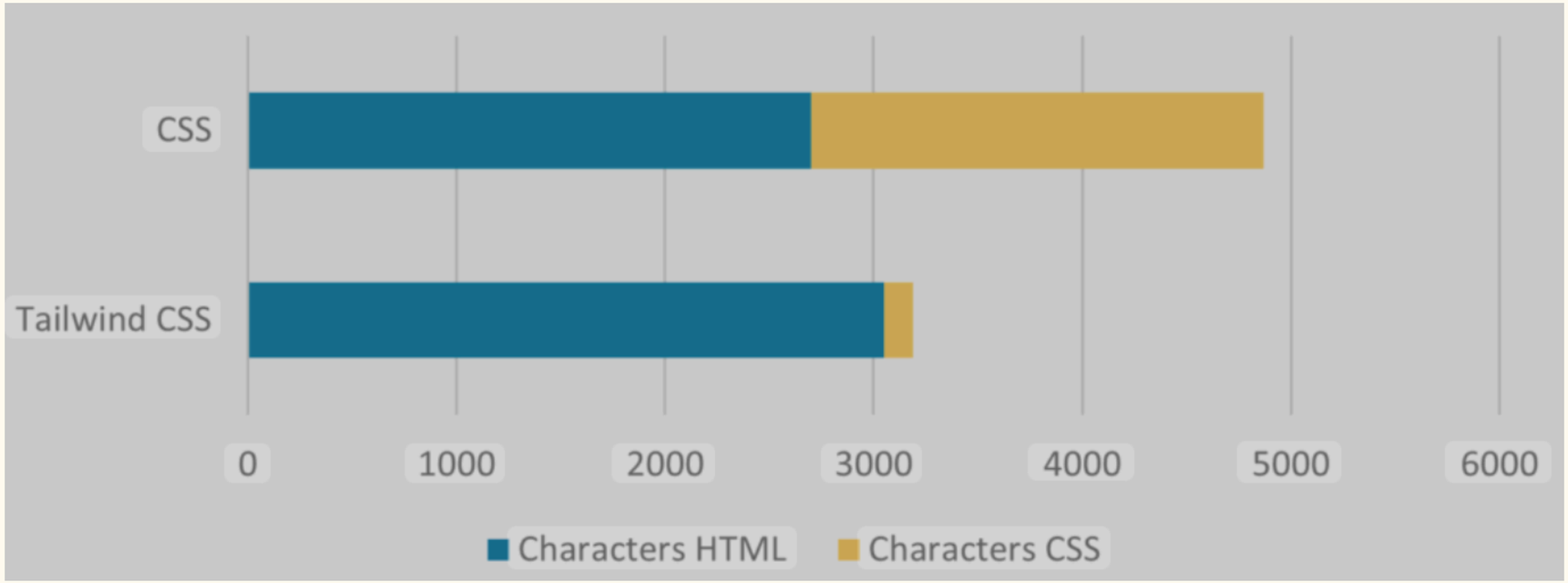
USING TAILWIND

Tailwind CSS




VS
Tailwind CSS
Bootstrap
Released on November 2017
Utility first
Highly customizable
Bloats HTML Template
Known for flexibility
Can reduce file size using PurgeCSS
Ideal for projects that require lot of customization
Released on August 2011
Component first
Less customizable
Keeps HTML Template Clean
Known for responsiveness
Larger file size than Tailwind CSS
Use it if you want to just play around with common layouts
www.encircletechnologies.com
support@encircletechnologies.com
ENCIRCLE
TECHNOLOGIES

Reduce code size



package.json

```
{
  "scripts": {
    "watch:css": "tailwindcss -i input.css -o output.css --watch"
  },
  "dependencies": {
    "@tailwindcss/cli": "^4.0.17",
    "tailwindcss": "^4.0.17"
  }
}
```

HTML head

```
<link data-trunk rel="css" href="output.css" />
```

In the view macro

```
view! {
    <div class="p-4 flex flex-col gap-2">
        <p class="text-xl font-bold">"Hello, Rustify! 🦀"</p>
        <button class="bg-sky-500 rounded-md p-2">"Button"</button>
    </div>
}
```

Exercice 2:
Using Tailwind

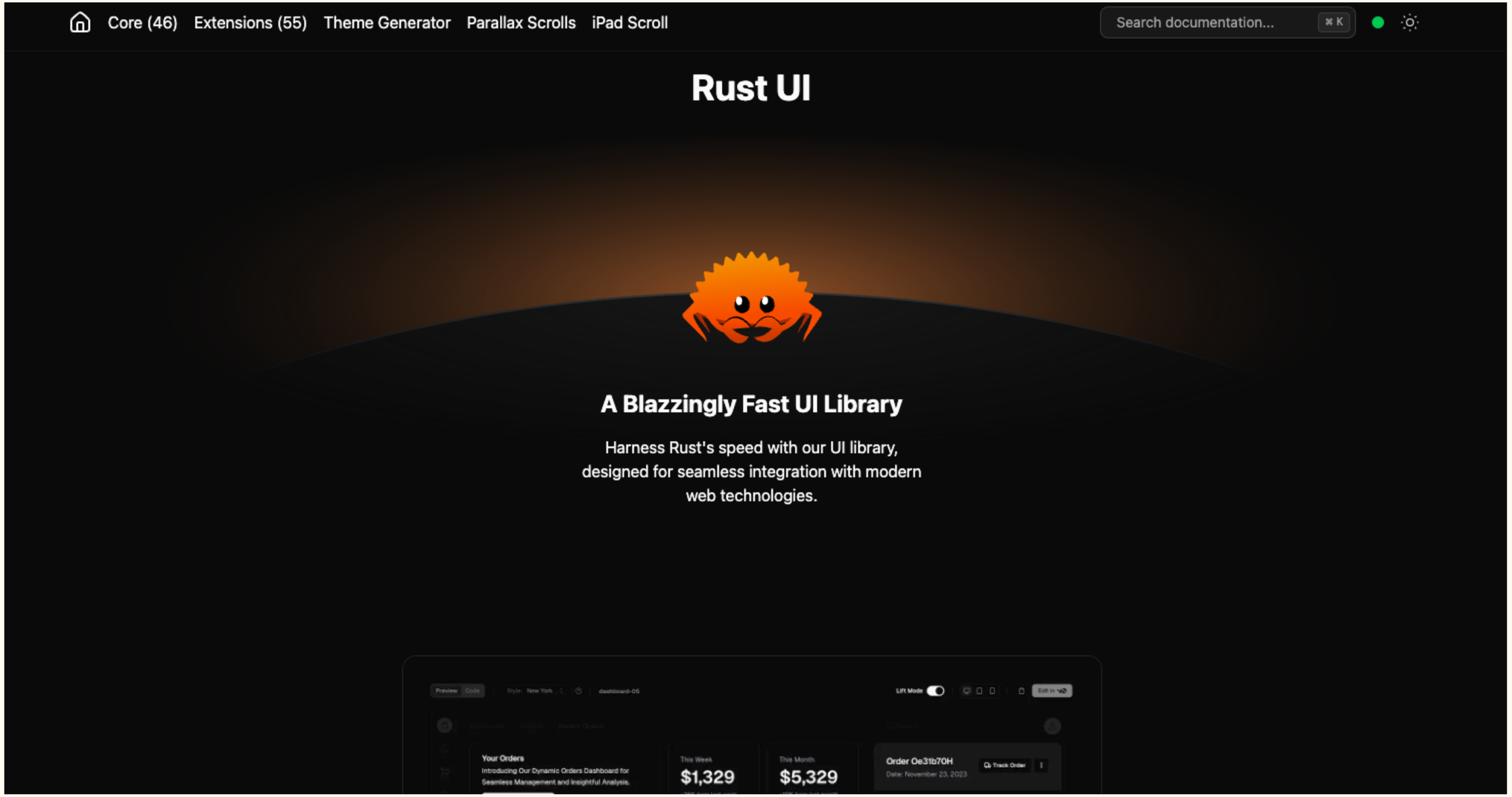
USE CLX! WITH TAILWIND

```
cargo install ui-cli --force
```

```
clx! {MyButton, button, "bg-sky-500 p-2 rounded-md"}

view! {
    <MyButton>"My Button"</MyButton>
}
```

www.rust-ui.com



Exercice 3:
Create components with clx! macro

Your turn!